Prezime i ime učenika	:	*Stoiljković Mihajlo*
Odeljenje i grupa	:	*IV-4 grupa 3.*
R. broj i naziv vežbe	:	*Vežba 11. Očitavanje jednog temperaturnog senzora, očitavanje više senzora istovremeno; Smeštaj analiza i vizuelizacija očitanih podataka, prikaz na veb interfejsu*

IZVEŠTAJ:

Cilj ove vežbe jeste analiza i vizuelizacija podataka uz pomoć python biblioteka i prikaz tih podataka na web interfejsu korišćenjem DHT11 senzora za vlažnost vazduha i temperaturu. Podatke koji se obrade se skladište u bazu podataka.

Struktura samog projekta treba da izgleda kao:

sensor_lab/

├── sensors.py # očitavanje senzora

├── database.py # rad sa bazom

├── webapp.py # Flask web server

├── data.db # SQLite baza

└── static/

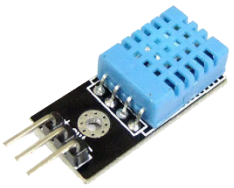
└── graph.png # grafikon

DHT senzori

DHT-xx (11 , 21) je senzor za merenje temperature i vlažnosti vazduha preko 1-wire komunikacije (jednožični digitalni interfejs). Senzor je fabrički kalibrisan tako da nema potrebe za dodatnim podešavanjima, već se može odmah primeniti u praksi. Sastoji se od kapacitivnog senzora vlažnosti vazduha (RH). Pored toga sastoji se od termistora za merenje temperature i dodatne elektronike za komunikaciju sa mikrokontrolerom. DHT senzori poseduju tri ili četiri pina (VCC,GND,data pin, otvoreni pin).

Za obezbeđivanje linije podataka i komunikacije između senzora i mikrokontrolera se koriste PULL UP otpornici 5-10K. Ovi senzori poseduju protokol za transfer podataka.

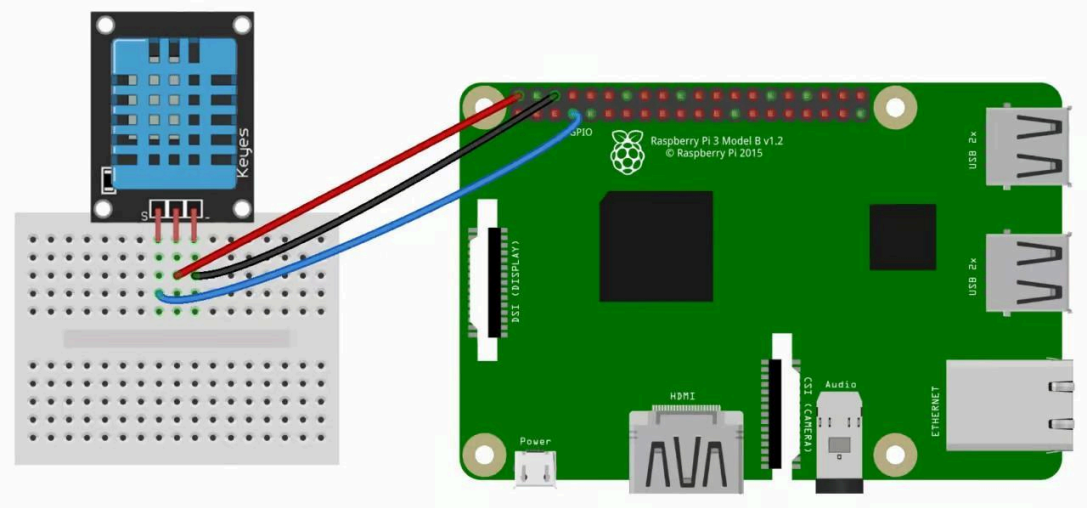
DHT11 je kapacitivni senzor relativne vlažnosti vazduha i temperature. Sastav:



- termistor → meri temperaturu (otpornost se menja sa temperaturom),
- kapacitivni senzor vlage → meri vlažnost (više vlage = manji otpor).

Ima otvorene bakrene vodove koji su međusobno vrlo blizu. Što je više vlage u vazduhu,više

vlage dolazi i na same vodove te je otpor među njima manji. Senzor čita dve vrednosti i pretvara ih u digitalni signal. Otpor se pojavljuje tek kada procenat vlažnosti vazduha (RH) predje 20%. Način povezivanja je prikazan na slici ispod.



Očitavanje dva DHT11 senzora

Prva skripta jeste ona koja očitava vrednosti sa DHT11 senzora i ispisuje u shell. U ovoj skripti se koriste moduli adafruit_dht kao i board pa je potrebno da se instaliraju s obzirom da ne dolaze uz python. Instalacija ovih biblioteka potrebno je provesti uz niz komandi koje idu jedna posle druge:

Pokretanje azuriranja za Raspberry Pi OS:

```
mihajlo@DESKTOP-BAFHUP9:~$ sudo apt update
```



Instalacija biblioteka za python verziju 3:

```
mihajlo@DESKTOP-BAFHUP9:~$ sudo apt install python3-pip -y
```

Instalacija `adafruit-blinka` modula:

```
mihajlo@DESKTOP-BAFHUP9:~$ pip3 install adafruit-blinka
```

Instalacija `adafruit-circuitpython-dht` modula:

```
mihajlo@DESKTOP-BAFHUP9:~$ pip3 install adafruit-circuitpython-dht
```

Nakon instalacija modula možemo napisati test skriptu u kojoj proveravamo da li modul board radi. Cilj test skripte jeste da printujemo D4 i D17 pinove pomoću board modula:

```
import board
print(board.D4)
print(board.D17)
```

Ukoliko nema greške, onda je sve instalirano kako treba, ako se greška javi, potrebno je proveriti i odraditi korake napisati iznad ispocetka. Nakon instalacija modula sledi python skripta za očitavanje vrednosti sa DHT11 senzora:

```
import time
import board
import adafruit_dht
# Definisanje dva DHT senzora
dht1 = adafruit_dht.DHT11(board.D4)
dht2 = adafruit_dht.DHT11(board.D17)
while True:
    try:
        # Citanje podataka sa senzora 1
        t1 = dht1.temperature
        h1 = dht1.humidity
        # Citanje podataka sa senzora 2
        t2 = dht2.temperature
        h2 = dht2.humidity
        # Stampanje podataka u shell-u
        print("Senzor 1 -> T:", t1, "H:", h1)
        print("Senzor 2 -> T:", t2, "H:", h2)
        print("----------------------")
        # Delay od 5 sekundi radi boljeg citanja
        time.sleep(5)
    except RuntimeError:
        pass
```

Na početku skripte importujemo module time, board i adafruit_dht. Modul time je predefinisan python modul i služi za rad sa vremenom tokom izvršavanja programa (kreiranje pauze, kašnjena i slično). Board modul služi lakši rad sa pločom (obezbeđuje simbolička imena GPIO pinova) čime se poboljšava čitljivost i prenosivost koda kao i adafruit_dht modul koji služi za rad sa DHT senzorima koji poseduje klase, metode i atribute koji olakšavaju rad očitavanja podataka sa senzora. U daljem delu programa definišemo dva DHT senzora i u beskonačnoj petlji odnosno dok program radi čitamo podatke sa oba senzora (vlažnost vazduha i temperaturu) i štampamo vrednosti u tonny IDE shell-u. Na kraju petlje imamo pauzu od 5 sekundi radi realnog i lakšeg očitavanja podataka sa senzora (svakih 5 sekundi se vrši iteracija petlje). Takodje, skripta poseduje i obradu izuzetaka koji ovde nisu konkretno definisani.

Skripta za smeštaj podataka (Python + SQLite)

U ovom delu projekta, koristimo SQLite bazu podataka pa ćemo reći nešto o njoj.

SQLite je embedded relaciona baza podataka koja ne zahteva poseban server niti ikakvu dodatnu infrastrukturu za rad s njom. Cela baza se čuva u jednom fajlu na disku što je čini pogodnom za neke lokalne projekte, IoT sisteme i razvoj aplikacija. Sve što napravimo bilo da su tabele, zapisi podatke i strukturu same baze se čuvaju u jednoj datoteci. Zanimljivost SQLite-a je da on nije samo jedan mali alat već ga koriste i velike



kompanije kao što su Apple, Google, Adobe i Mozilla, jer im omogućava da unutar svojih softvera imaju stabilnu bazu podataka bez potrebe za dodatnim serverima.



Za instalaciju SQLite-a na RaspberryPi OS-u, potrebno je uneti dve komande:

Ulazak u azuriranje OS-a:

```
mihajlo@DESKTOP-BAFHUP9:~$ sudo apt update
```

Instalacija sqlite3 modula:

```
mihajlo@DESKTOP-BAFHUP9:~$ sudo apt install sqlite3
```

Unutar samog SQLite-a radimo sve kroz SQL komande I upite. Vrlo je sličan sintaksi MySql-a. Nama će ova baza biti potrebna za čuvanje podataka sa dva senzora DHT11. Python skripta za smeštaj podataka:

```
import sqlite3
from datetime import datetime
conn = sqlite3.connect("data.db")
# Definisanje promenljive koja koristi nasu konekciju baze
c = conn.cursor()

# Kreiranje tabele dht_data ukoliko ne postoji
c.execute("""
CREATE TABLE IF NOT EXISTS dht_data(
    time TEXT,
    t1 REAL,
    h1 REAL,
    t2 REAL,
    h2 REAL
```

```
)
""")
def save(t1, h1, t2, h2):
    c.execute("INSERT INTO dht_data VALUES (?, ?, ?, ?, ?)",
        (datetime.now(), t1, h1, t2, h2))
    conn.commit()
```

Na početku skripte, importujemo modul sqlite3 i modul datetime koji služi za rad sa datumom i vremenom (čuvanje vremena merenja). Nakon toga pravimo konekciju sa SQLite bazom podataka data.db fajla. Ukoliko ta baza podataka ne postoji, on je automatski kreira u našem folderu projekta. Važno je napomenuti da se ne dodaje ovaj fajl pre kreiranja baze, jer će se fajl automatski dodati nakon što kroz skriptu kreiramo samu bazu. Nakon toga, kreiramo kursor koji omogućava izvršavanje SQL komandi nad bazom podataka. Ispod toga izvršavamo SQL komandu unutar **execute()** metode gde kreiramo tabelu ukoliko ne postoji pod imenom **dht_data**. Ova tabela se koristi za smeštaj:

- vremena merenja (time)
- temperature i vlažnosti vazduha sa prvog senzora (t1, h1)
- temperature i vlažnosti vazduha sa drugog senzora (t2, h2)

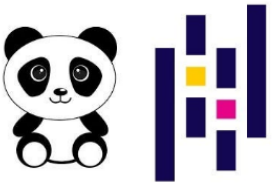
Korišćenjem uslova `IF NOT EXISTS` sprečavamo ponovno kreiranje baze nakon svakog pokretanja skripte odnosno programa. Nakon toga sledi funkcija **save** koja služi za čuvanje podataka i upisivanje istih u bazu podataka odnonso u tabelu dht_data. Korišćenjem parametarskog upita **?** se povećava bezbednost I sprečavaju se SQL greške. Metoda **commit()** potvrđuje I trajno upisuje promene u bazu.

Skripta za analizu podataka (Pandas)

Što se tiče analize podataka, kao deo projekta je vrlo važno obraditi podatke iz baze podataka s obzirom da vršimo crtanje grafikona na web interfejsu odnosno u web aplikaciji. Paket predstavlja skup modula dok biblioteka predstavlja skup modula I paketa.

Pandas je python biblioteka namenjena za obradu, analizu I manipulaciju podataka. Najčešće se koristi za rad sa tabelarnim podacima, sličnim tabelama ili bazama podataka. Osnovna struktura podataka u Pandas biblioteci je **DataFrame** koji predstavlja tabelu sastavljenu od redova i kolona. Svaka kolona ima ime I može sadržati različite tipove podataka. U našem projektu, pandas koristimo za učitavanje podataka iz SQLite baze, statističku analizu podataka kao i poređenje merenja sa više senzora.

Prednost **Pandas** biblioteke je:



- jednostavna analiza velikog broja podataka
- brza statistička obrada
- laka integracija sa bazama podataka

Pandas biblioteka nije predefinisana u python, pa je potrebno naknadno instalirati komandom:

```
mihajlo@DESKTOP-BAFHUP9:~$ pip3 install pandas
```

Nakon instalacije modula, sledi python skripta za analizu podataka iz naše baze podataka **data.db:**

```
import pandas as pd
# kreiranje DataFrame-a
df = pd.read_sql("SELECT * FROM dht_data", conn)
# Stampanje podataka
print("Prosečna temp (S1):", df['t1'].mean())
print("Prosečna temp (S2):", df['t2'].mean())
print("Razlika temperatura:", (df['t1'] - df['t2']).mean())
```

Na početku skripte importujemo modul pandas, zatim kreiramo DataFrame odnosno tabelu i podaci iz naše baze podataka se učitavaju u **DataFrame** gde:

- svaki red predstavlja jedno merenje,
- svaka kolona predstavlja jednu veličinu (vreme, temperatura, vlažnost).

Recimo **df[‘t1’]** predstavlja temperature sa prvog senzora i slično. Funkcija mean() je Pandas funkcija za izračunavanje aritmetičke sredine numeričkih podataka. Naravno unutar promenljive za učitavanje DataFrame-a imamo SQL upit gde selektujemo sve iz tabele dht_data i predstavljamo odnosno učitavamo u **DataFrame** za analizu podataka.

Skripta za vizuelizaciju podataka (Matplotlib)

U prethodnom delu projekta smo vršili analizu podataka a u ovom delu ćemo vršiti vizuelizaciju podataka kroz jedan grafikon. Ovo postižemo uz pomoć python biblioteke matplotlib

Matplotlib je popularna python biblioteka koja služi za vizuelizaciju podataka. Omogućava kreiranje raznih tipova grafova: linijskih, stubičastih, kružnih... Najčešće se koristi modul pyplot za crtanje 2D grafova i radi na principu crtanja linija i nekih legendi i oznaka. U našem projektu, u ovom delu nam je cilj da ostvarimo prikaz temperature sa dva DHT11 senzora u zavisnosti od vremena.



Pre nego što napišemo skriptu, potrebno je da instaliramo uz pomoć linux terminala ovu biblioteku komandom:

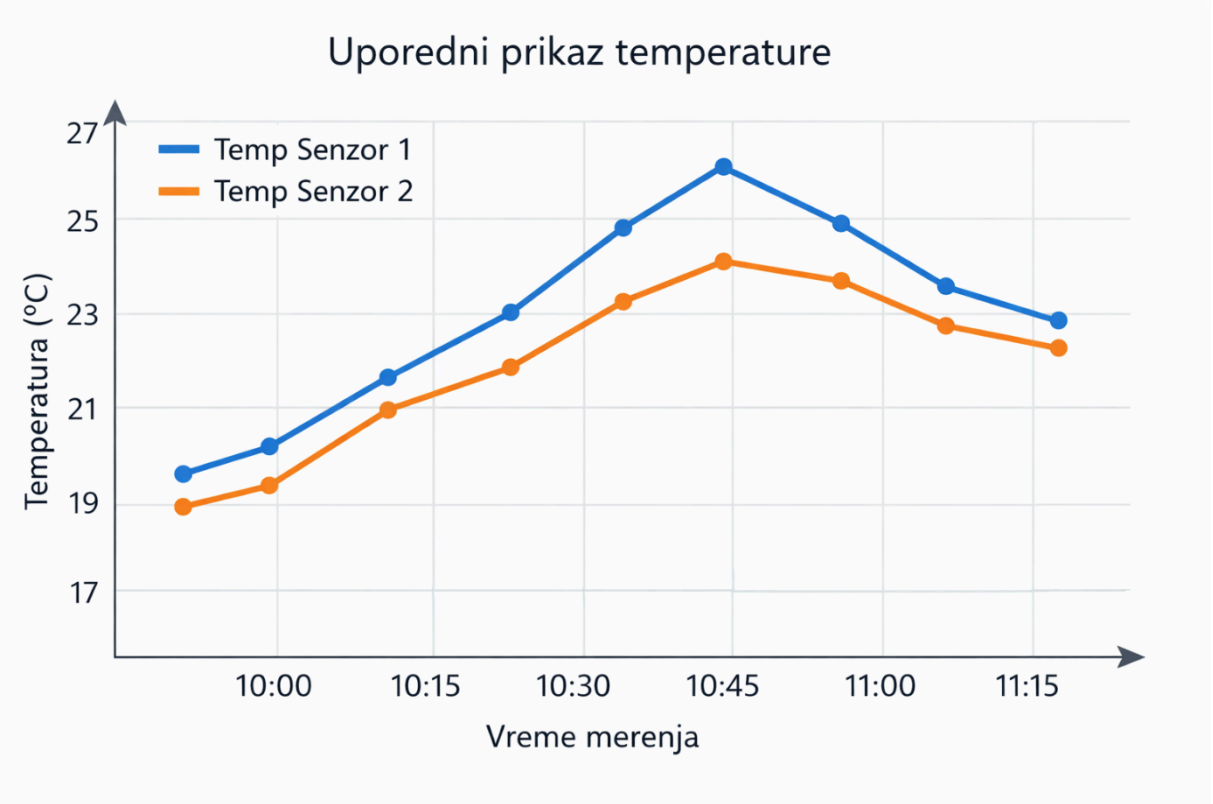
```
mihajlo@DESKTOP-BAFHUP9:~$ pip3 install matplotlib
```

Nakon instalacije, možemo napisati skriptu za crtanje grafikona:

```
import matplotlib.pyplot as plt
plt.plot(df['time'], df['t1'], label="Temp Senzor 1")
plt.plot(df['time'], df['t2'], label="Temp Senzor 2")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
```

```
plt.savefig("static/graph.png")
```

Na početku importujemo biblioteku matplotlib i konkretno njen modul pyplot za 2D grafikon. Nakon toga vršimo crtanje linijskog grafa sa x i y vrednostima uz pomoć **plot**() metode. Metoda **legend**() dodaje legendu za linije na grafu, **xticks**() metoda rotira oznake na X osi za 45 stepeni, **tight_layout**() metoda automatski podešava razmeke na grafu da ništa ne bude isečeno i na kraju **savefig**() metoda vrši čuvanje fajla na datoj lokaciji odnosno prema datoj navigaciji (u našen slučaju u folderu projekta static kao graph.png). Primer kako ovaj grafikom treba da izgleda je dat na slici ispod:



Kreiranje web aplikacije (Flask)

Nakon kreiranja skripti za očitavanje, obradu, smeštaj, analizu i vizuelizaciju podataka, vreme je da kreiramo aplikaciju unutar koje te podatke trebamo da prikazemo. Za ovo ćemo koristiti web framework Flask.

Framework (okvir) je skup biblioteka, modula i pravila koji olakšava razvoj aplikacija što daje gotove funkcionalnosti i strukturu projekta. Predstavlja jedan alat sa upustvom, umesto da sve pravimo od nule, on daje osnovu i pravila a mi dodajemo našu funkcionalnost za projekat.

Flask je python web framework za razvoj Web aplikacija i web servera. Omogućava da uz pomoć pythona napravimo web interfejs za prikaz podataka i slično. Nudi minimalističku strukturu, omogućavajući nam da



dodamo samo potrebna proširenja prema projektu. Programeri ga često biraju zbog svoje strukture i podrške za integraciju sa bazama podataka. Podržava dodatke (plugins) za baze, autentifikaciju, REST API i slično. Flask web aplikacija je data ispod:

```
from flask import Flask, render_template_string

app = Flask(__name__)

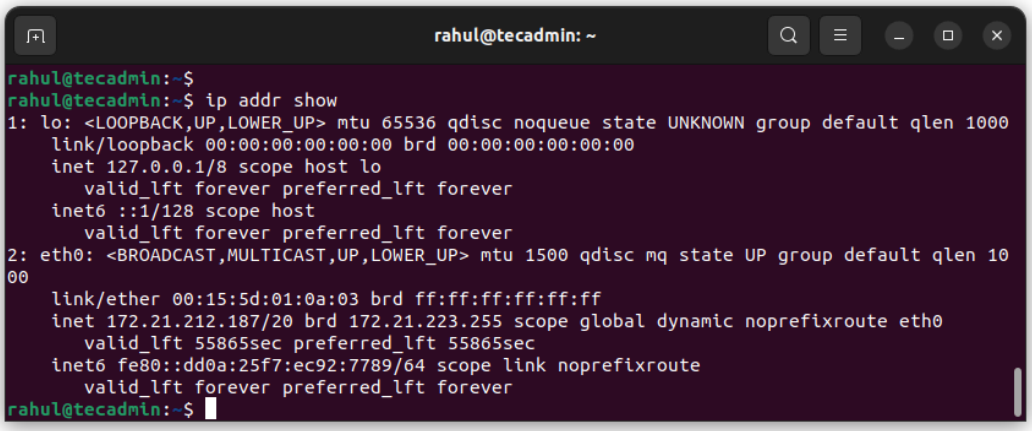
# definisanje HTML koda
HTML = """
<h1>DHT11 - Poređenje senzora</h1>
<img src="/static/graph.png">
"""
@app.route("/")
def index():
    return render_template_string(HTML)

# Pokretanje aplikacije
app.run(host="0.0.0.0", port=5000)
```

Na početku importujemo modul Flask kao i render_template_string koji služi za prikaz HTML sadržaja. Nakon toga definišemo promenljivu koja predstavlja aplikaciju i gde koristimo Flask klasu za pokretanje. Unutar promenljive **HTML** definišemo osnovni **HTML** kod gde imamo jedan heading tag i sliku odnosno grafikon koju smo kreirali iz pomoć matplotlib-a. Mogli smo to izvršiti i preko zasebnog HTML fajla ali nije bilo potrebe. Nakon toga koristimo Flask dekorator **@app.route()** koji služi za definisanje stranice u kojoj će se vršiti prikaz. Konkretno **“/”** definište **HOME** stranicu unutar flask aplikacije. U metodi index vraćamo HTML web stranicu uz pomoć render_template_string() metode. Na kraju vršimo pokretanje aplikacije na adresu 0.0.0.0 i portu 5000. Adresa 0.0.0.0 znači da server sluša na svim mrežnim interfejsima Raspberry Pi-a. To uključuje lokalni interfejs odnosno **localhost** kao i lokalnu mrežu na kojoj se nalazi Rpi. Bilo koji uredjaj na istoj mreži može da pristupi serveru preko ip adrese Rpi-a i porta **5000**. Pristup iz web browsera ide kao:

http://IP_ADRESA_RPi:5000

Ukoliko ne znamo ip adresu Raspberry Pi-a, unutar terminala unosimo komandu:



Ovde gledamo **eth0** port I inet ip adresu. Primer izgleda web aplikacije je prikazan na slici:

